YuMi Drawing Robot User Manual

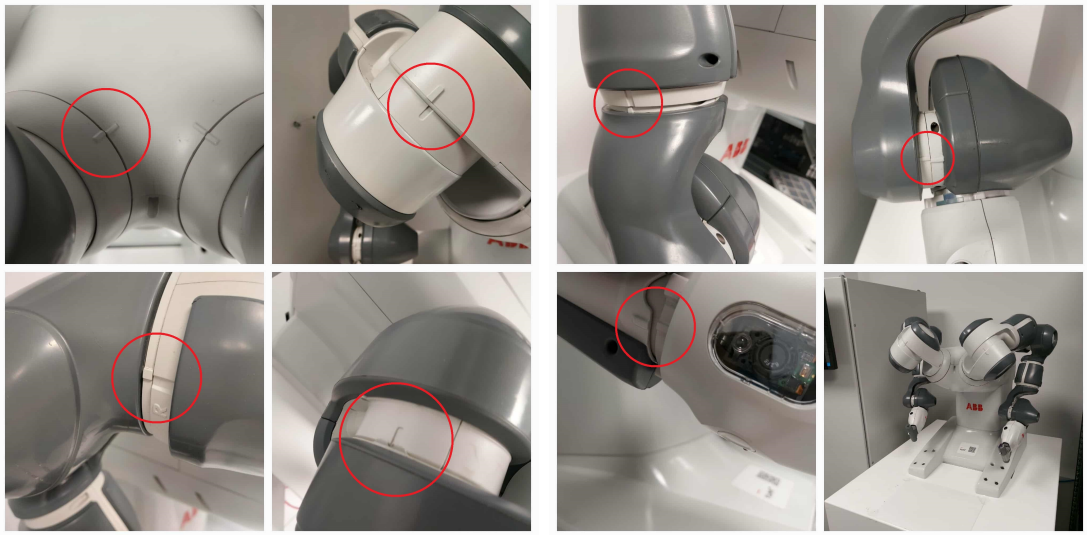
Contents

1 Step-by-Step Guide

1.1 Preparation for Calibration

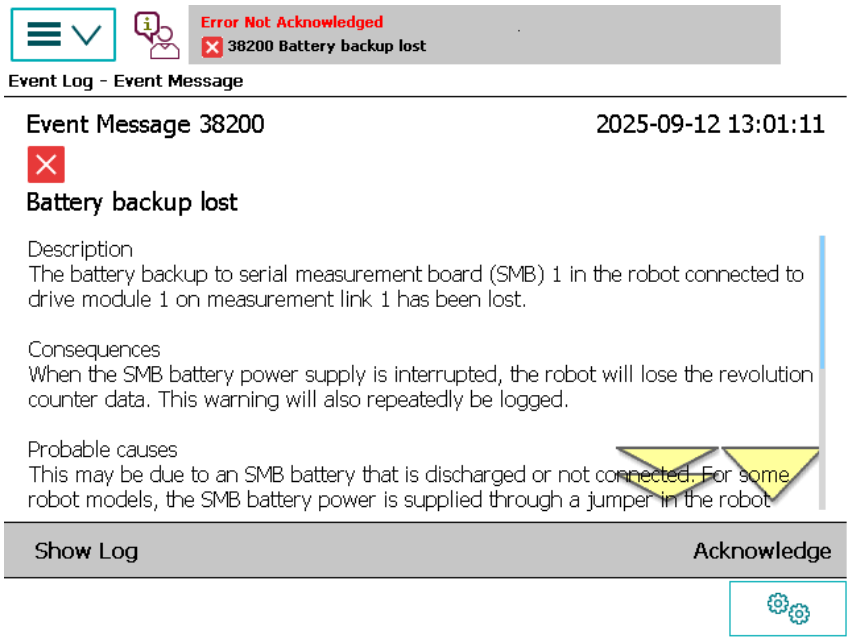
1.1.1 Robot Positioning

With the robot turned off, position it for calibration as shown in the pictures (visible teeth and indicators).



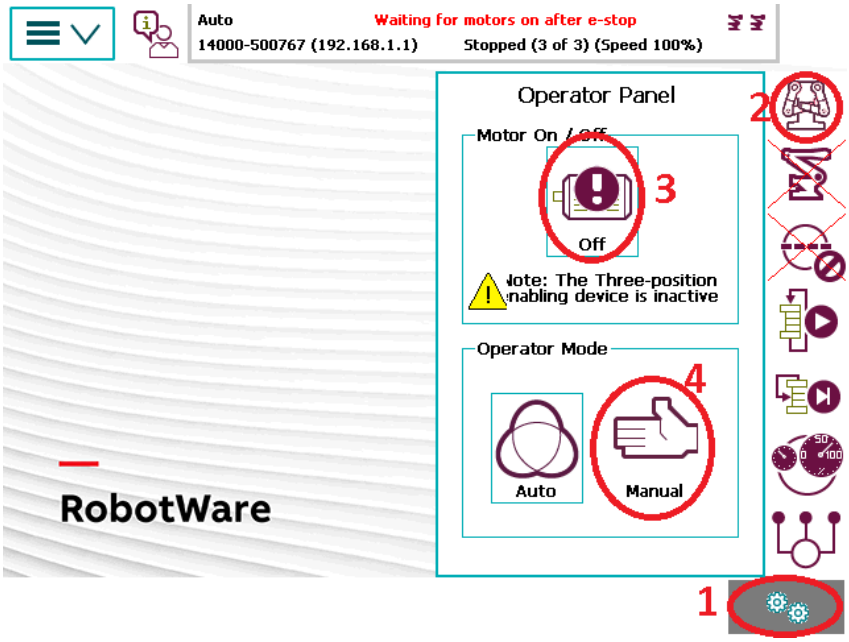
1.1.2 Error Handling

Acknowledge all errors.



1.1.3 Switching to Manual Mode

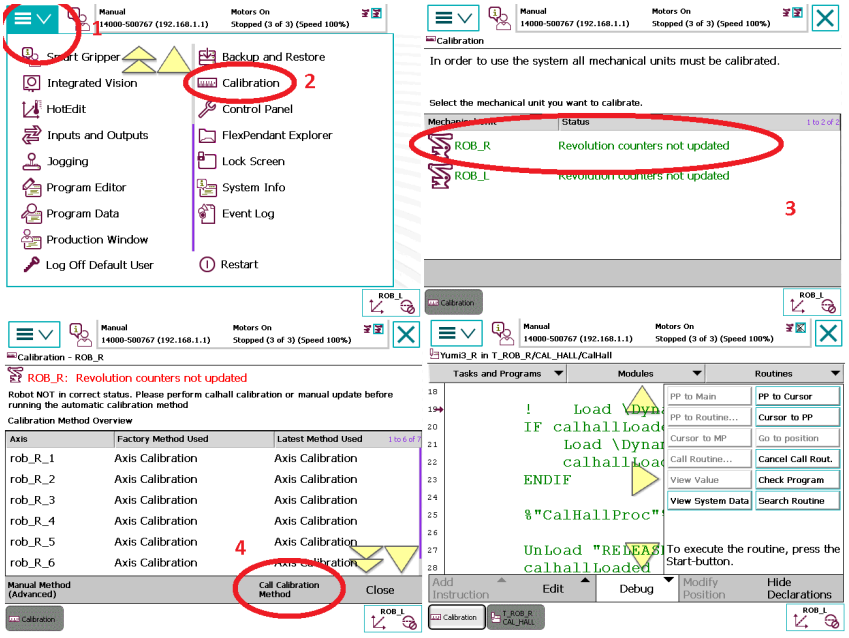
Start the motors and switch to manual mode. If errors pop up, accept them and try again.

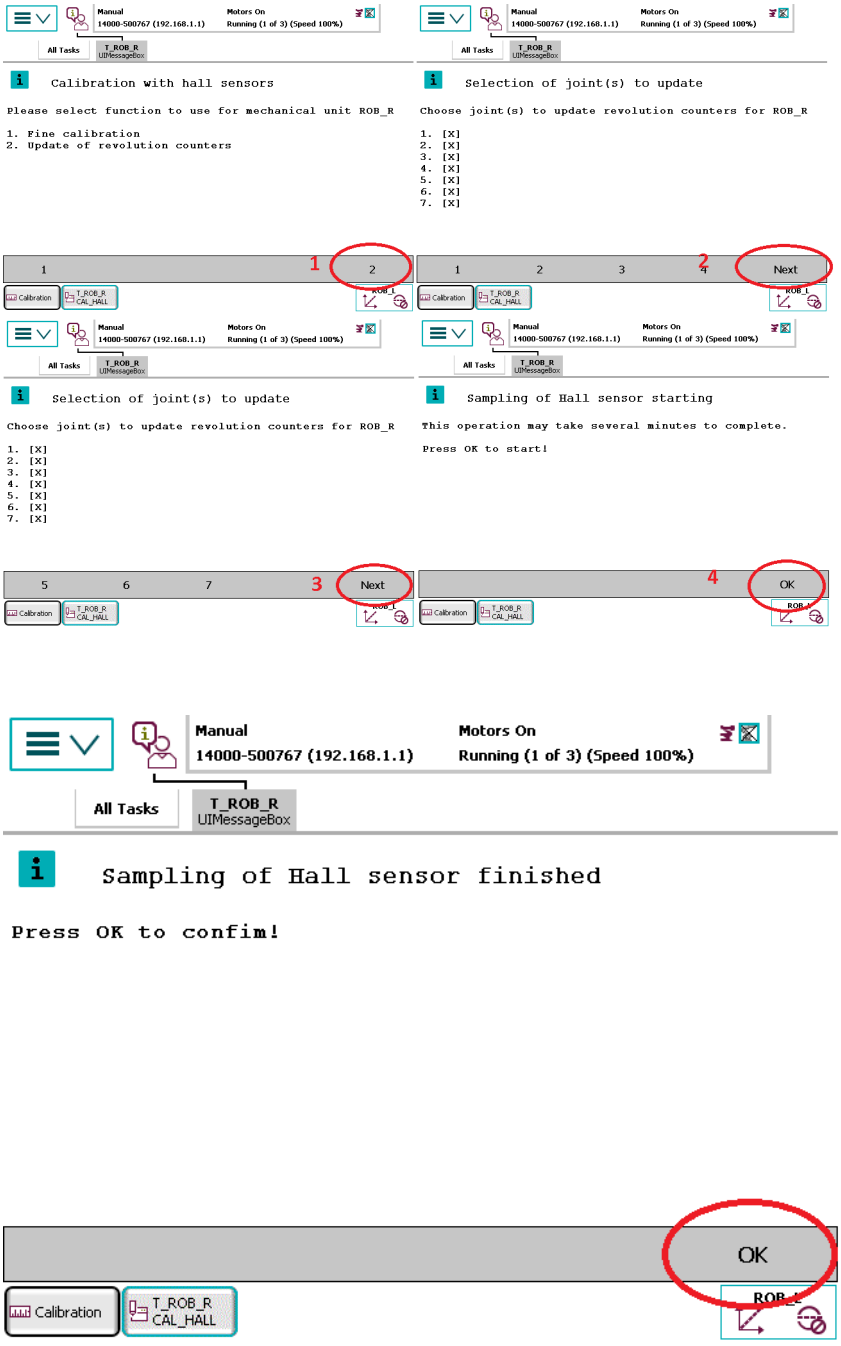


1.2 Arm Calibration

1.2.1 Calibrating the First Arm

Initiate the calibration for the first arm. All pop-up errors should be acknowledged. Then, press the triangle-marked button on the flexpendant to start the application.



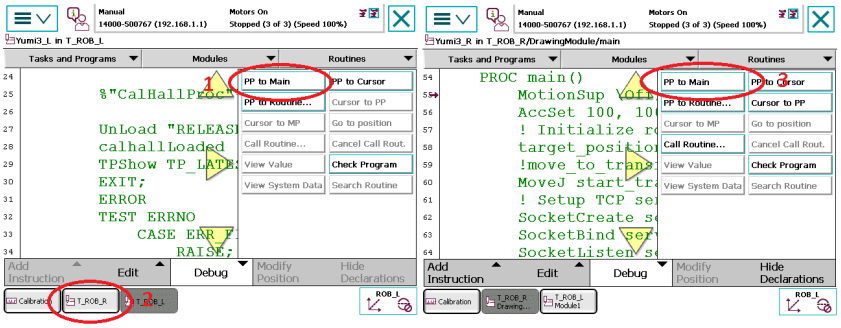


1.2.2 Calibrating the Second Arm

Initiate the calibration for the second arm, similar to the first one.

1.2.3 Pointer to Main

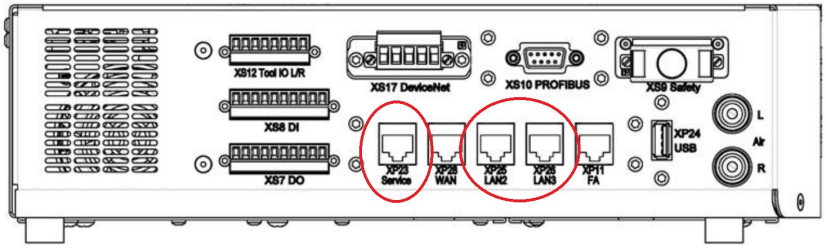
Set the **pointer to main** for both arms.



1.3 Connecting to the Computer and AUTO Mode

1.3.1 Connection

Connect the robot's LAN port to the computer using an Ethernet cable.



1.3.2 Automatic Mode

Switch to automatic mode (the same way you switched to manual, but without starting the motors).

1.4 Starting the Application

1.4.1 On the Robot

Launch the application on the robot. Press the triangle-marked button on the flexpendant to start the application.

Note: The robot will start moving!

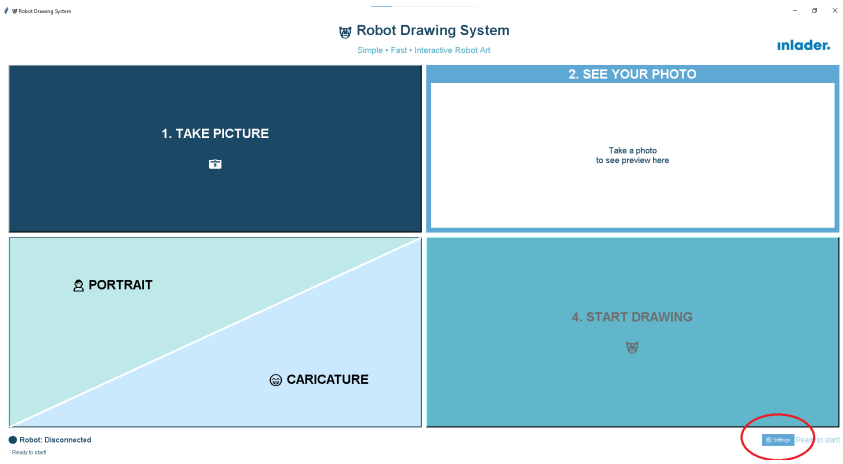
1.4.2 On the Computer

Launch the application on the computer by opening the `simple_gui.py` file in Python.

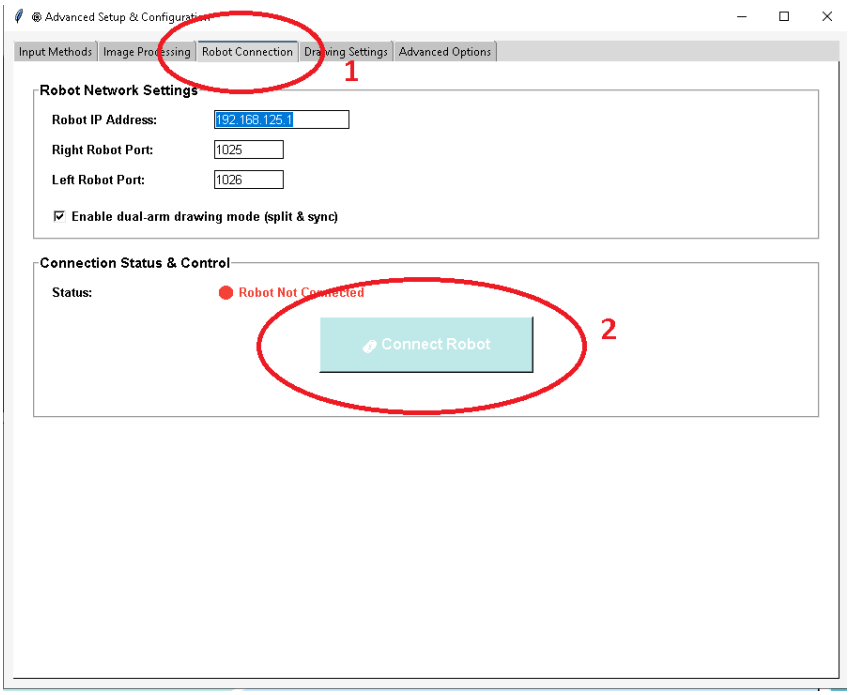
1.5 Operating the Graphical Application

1.5.1 Connection Settings

Go to **Settings**.



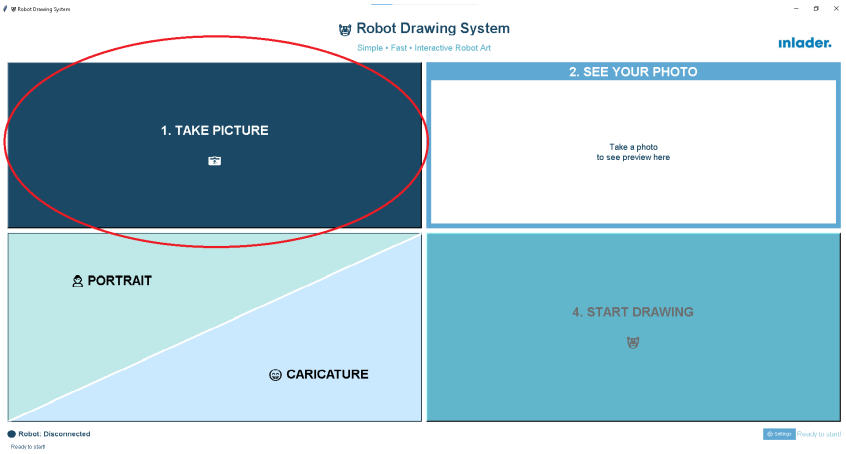
In the **Robot Connection** section, click **Connect**.



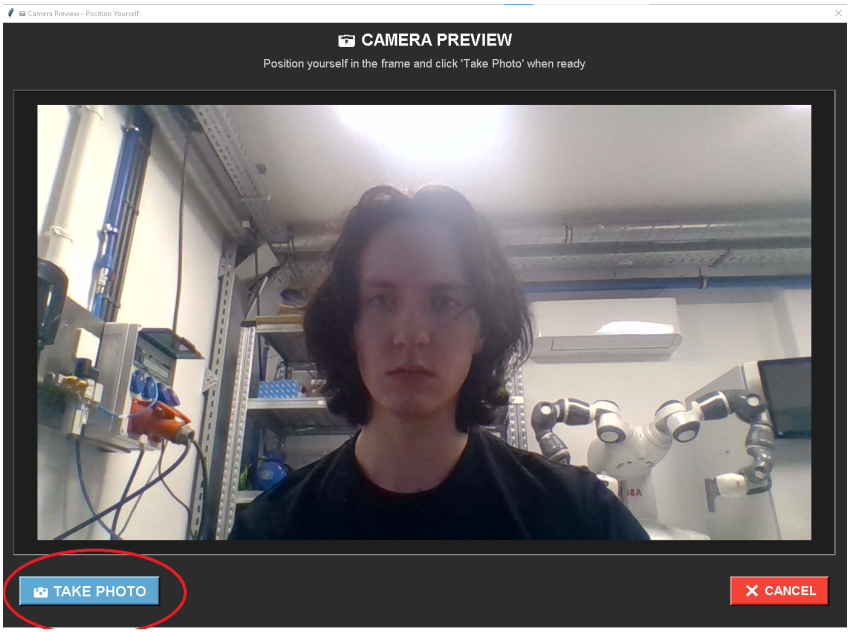
Close the settings.

1.5.2 Taking a Photo

Click **Take Picture**.



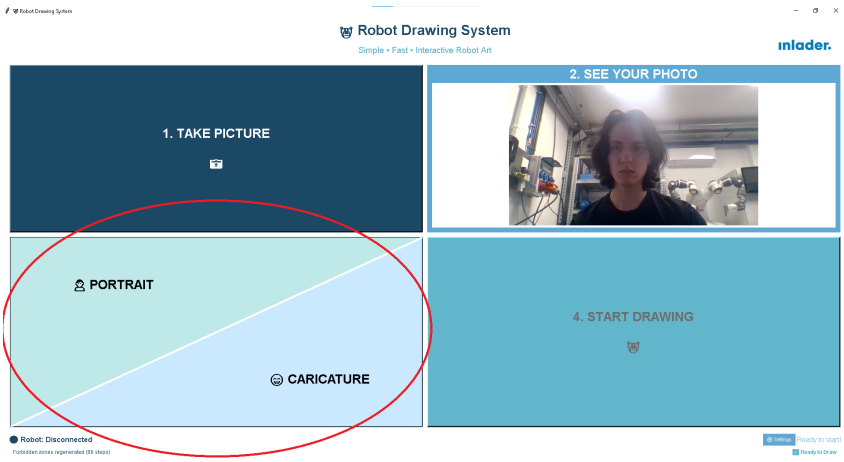
Position yourself and click **Take Photo**.



Evaluate the photo. If it doesn't meet expectations, take a new one.

1.5.3 Selecting a Mode

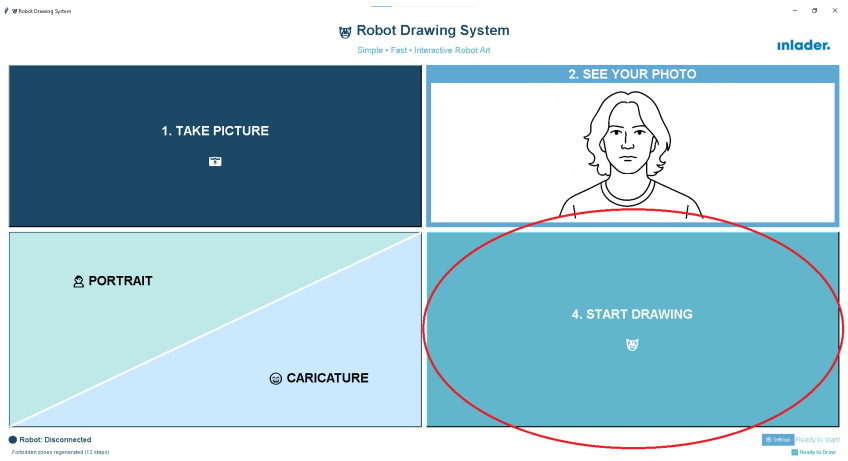
Choose a mode: **Portrait** or **Caricature**.



The image will be generated (requires a stable internet connection).

1.5.4 Starting to Draw

Press **Start Drawing** to begin the drawing process.



1.6 Voice Commands

- **Kamera** - turn on the camera
- **Akcja** - take a photo
- **Portret** - create a portrait
- **Karykatura** - create a caricature
- **Połącz** - connect to the robot
- **Start** - start
- **Stop** - stop

2 Most Common Errors and Solutions

2.1 Robot Connection Problems

Error: Unable to connect to the robot. The application displays a connection error message.

Solution:

- Check if the robot is turned on and in automatic mode.
- Make sure the Ethernet cable is correctly connected between the computer and the robot.
- Verify the robot's IP address (default is 192.168.125.1) and ports (1025 for the right arm, 1026 for the left).
- Check your computer's firewall settings ensure that TCP connections on ports 1025 and 1026 are allowed.
- Restart the application on both the robot and the computer.

2.2 Calibration Errors

Error: The robot doesn't calibrate or displays errors when the application starts.

Solution:

- Make sure the robot is in the calibration position (visible teeth and indicators in the photos).
- Acknowledge all errors on the robot panel using the "Acknowledge errors" button.
- Switch to manual mode, then return to automatic mode.
- Set the "pointer to main" on both arms.

2.3 Camera Problems

Error: Unable to take a picture or the camera isn't working.

Solution:

- Check if the camera is connected to the computer and recognized by the system (e.g., in Windows Device Manager).
- Make sure no other applications are using the camera.
- Try changing the camera index in the application settings (default is 0).
- If it's an external camera, check the USB connection and power supply.
- Close and restart the application.

2.4 OpenAI Image Generation Errors

Error: OpenAI doesn't generate an image or returns an API error.

Solution:

- Check your internet connection a stable link is required.
- Ensure that the OpenAI API key is correctly configured in the application.
- Check the API usage limit if it's exceeded, wait or top up your account.
- Try again with the same photo or take a new one.
- If the error persists, check the status of OpenAI services.

2.5 Image Processing Problems

Error: The processed image has poor contours or is unreadable.

Solution:

- Try a different bineralization detection method (threshold, canny, or adaptive) in the settings.
- Fine-tune the quality parameters (higher quality for better results, but longer processing time).
- Take a better photo ensure good lighting and sharpness.
- Check if the image is too complex, simplify the background or position.

2.6 Drawing Errors

Error: The robot stops while drawing or a collision occurs.

Solution:

- Switch to manual mode and move the arms to a safe location.
- Increase the collision buffer in the settings (default is 40 mm).
- Check the safety margins (default is 10 mm from the edge).
- If a timeout occurs, increase the timeout in the settings or check the connection.
- In case of a stop, use the emergency stop button on the robot.

2.7 Voice Recognition Problems

Error: VOSK doesn't recognize voice commands.

Solution:

- Make sure the VOSK model (vosk-model-small-pl-0.22) is downloaded and in the correct folder.
- Speak clearly in Polish, in a quiet environment.

- Adjust the microphone sensitivity in your system settings.
- If it doesn't work, use text commands instead of voice.

2.8 Other Problems

Error: The application freezes or displays Python errors.

Solution:

- Check the terminal for any errors.
- Verify that all required libraries are installed (e.g., OpenCV, Matplotlib, Shapely).
- Run the application from the command line to see detailed errors.
- Check RAM availability processing large images requires resources.
- Update Python to version 3.8+.
- If the error persists, restart the computer and try again.

3 Alarms to Acknowledge

Press "Acknowledge" whenever they appear.

- **20032** Rev counter not updated
- **38200** Battery backup lost
- **50091** Restart not possible
- **50441** Low voltage on battery inputs
- **50490** Measurement error detected
- **71367** No communication with I/O device

4 FAQ

4.1 How does the entire drawing process work?

The whole process begins with a photo taken by a camera connected to the computer. This photo is then sent via the OpenAI API with a chosen prompt (portrait or caricature) to generate a new image in the selected style. The returned image is processed using one of three bineralization methods (threshold, canny, or adaptive), simplified to contours, rotated, and optimized for the drawing path. Finally, the ABB YuMi robot draws the image based on the generated coordinates.

4.2 What are the image processing methods?

The system offers three bineralization methods:

- - **Threshold**: Simple binarization of the image with a rigidly set brightness cutoff (good for good lighting). 1 where it's darker, 0 where it's lighter.

- **Canny**: Finds changes in brightness. Where there are changes, we get 1, where there are no changes, we get 0.
- **Adaptive**: The same as Threshold, but it automatically selects the parameters (more universal).

4.3 Why do we process images in the cloud with AI?

We can get much nicer, simpler lines to create the best image for drawing. Otherwise, the robot would work for an extremely long time to draw every shadow in the hair, etc.

4.4 How exactly do we convert a binary image to contours?

After applying one of the three bineralization methods (threshold, canny, or adaptive), the image becomes binary black and white, where white pixels represent edges and black is the background. We then use an algorithm that scans the image and identifies connected groups of white pixels as contours. Each contour is a list of coordinate points (x,y) that describe the shape of the line. The contours are simplified using ‘approxPolyDP()‘ with an epsilon parameter to reduce the number of points without losing the shape this is crucial for efficiency because the robot doesn't have to draw every single pixel. Finally, the contours are filtered: small artifacts are removed, and large contours are split into segments for better control.

4.5 How does creating the drawing path along the contour work?

When we have contours as lists of points, the system creates a drawing path, treating each contour as a route. The points are connected in a sequence, creating a continuous line. For fluidity, we use smoothing a Bézier algorithm interpolates the points, eliminating sharp corners. We can optionally enable TSP (Traveling Salesman Problem), which optimizes the order of drawing contours, minimizing the travel time between them (e.g., drawing the nearest contour next). The path is scaled to the robot's coordinates (e.g., 290x210 mm) with safety margins. In dual-arm mode, the paths are split between the arms, and each point is assigned a movement time for synchronization.

4.6 How does the system intelligently split work between two robot arms?

In dual-arm mode, the system uses an advanced master-slave assignment algorithm, but note the master and slave roles are not fixed! The system dynamically decides which arm becomes the "operation brain" (master) and which is the "support" (slave), depending on the contour's position, but they usually switch roles with each contour. Special zones are key: the left arm has a forbidden zone (0,0 to 130,40 mm), so it always avoids this area to prevent collision with the robot base. The algorithm uses a collision buffer (default 40 mm) a circular "safety bubble" around the master and a "tail" from this "bubble" towards the master's side of the paper, which the slave cannot enter. The algorithm checks for the overlap of contour polygons with these buffers, intelligently assigning tasks to avoid collisions. Additionally, contours are sorted by position: the closer arm gets priority. The arms travel in a sharp U-shape between contours to avoid collisions during

transitions and to limit the time the robots spend very close to each other. All this together allows for two-arm drawing.

4.7 How does communication between the computer and the robot work?

Communication between the computer and the ABB YuMi robot takes place via the TCP/IP protocol. The computer connects to the robot on two ports: 1025 for the right arm and 1026 for the left arm. Text commands are sent, such as "MOVE,x,y" to move an arm, "BATCH, x_1, y_1, x_2, y_2,..." to send a group of points, and "PEN UP" and "PEN DOWN" to control the tool. In dual-arm mode, arm synchronization is achieved using thread barriers and "WAIT" commands. The batch mode groups points into packets (maximum 15 points), which increases efficiency. The system handles errors through timeouts, retries, and disconnection detection. The robot confirms command execution with an "OK" response.

5 Detailed Code Structure

5.1 Main Application Files

5.1.1 simple_gui.py - User Interface

Location: Main project folder

Main Classes and Functions:

- `SimpleRobotGUI` - Main interface class
- `toggle_connection()` - Manages connection to the robot
- `_handle_command()` - Handles voice commands
- `convert_to_face_drawing()` - Conversion using AI
- `start_robot_drawing()` - Starts the drawing process

Key Configuration Variables:

- `robot_ip` - Robot's IP address (default "192.168.125.1")
- `robot_port` - Right arm's port (default 1025)
- `robot_port_l` - Left arm's port (default 1026)
- `max_x, max_y` - Working area dimensions (mm)
- `margin_x, margin_y` - Safety margins (mm)

5.1.2 robot_drawer.py - Main Coordinator

Location: Main project folder

Main Class:

- `RobotDrawer` - Main system orchestrator

Key Methods:

- `connect()` - Establishes a connection with the robot
- `load_image()` - Loads and processes an image
- `draw()` - Draws with a single arm
- `draw_dual()` - Draws with two arms
- `disconnect()` - Disconnects from the robot

5.1.3 robot_communication.py - TCP/IP Communication

Location: Main project folder

Main Class:

- `RobotController` - Manages the TCP connection

Key Methods:

- `connect()` - Establishes a socket connection
- `send_move(x, y)` - Sends a single movement command
- `send_batch_moves()` - Sends multiple movements
- `send_pen_up/down()` - Controls the pen
- `send_start()` - Starts the drawing

5.1.4 image_processor.py - Image Processing

Location: Main project folder

Main Functions:

- `load_and_process_image()` - Main processing function
- `detect_edges_threshold()` - bineralization using threshold method
- `detect_edges_canny()` - bineralization using Canny method
- `detect_edges_adaptive()` - Adaptive bineralization
- `find_contours()` - Finds contours
- `simplify_contours()` - Simplifies contours

5.1.5 coordinate_transformer.py - Coordinate Transformation

Location: Main project folder

Key Functions:

- `transform_paths()` - Main coordinate transformation
- `master_slave_assign_contours()` - Assigns contours to arms
- `smooth_path()` - Smooths paths

- `optimize_path_order()` - Optimizes order (TSP)

5.1.6 voice_commands.py - Voice Recognition

Location: Main project folder

Main Class:

- `VoiceCommandListener` - Handles voice commands

`convert_to_lineart.py` - OpenAI Integration **Location:** Main project folder

Main Functions:

- `generate_image_from_text()` - Generates an image from text
- `convert_to_lineart()` - Converts to a caricature or portrait

5.2 Configuration and Data Files

5.2.1 robot_gui_config.json - Application Configuration

Location: Main project folder

Example Content:

```
{
  "robot_ip": "192.168.125.1",
  "robot_port": "1025",
  "robot_port_l": "1026",
  "camera_index": 0,
  "use_center_origin": false,
  "enable_logo": true,
  "logo_size": 27,
  "enable_frame_filtering": true,
  "max_x": 290,
  "max_y": 210,
  "margin_x": 10,
  "margin_y": 10,
  "quality_var": "high",
  "detection_method": "adaptive",
  "enable_tsp": false,
  "use_batch_mode": true,
  "drawing_mode": "load",
  "brush_size": 20,
  "dual_arm_mode": true,
  "text_prompt": "",
  "forbidden_buffer": 50
}
```

5.2.2 line_art_catalog/ - Generated Image Archive

Location: Main project folder

Folder structure: `YYYYMMDD_HHMMSS_style/`

- Contains all generated images
- Automatically created during processing
- Useful for debugging and archiving

5.3 RAPID Files for the Robot

5.3.1 Right_arm.mod - Right Arm Program

Location: Main project folder

Key Parameters:

- Port: 1025
- Speed: v1500 mm/s

5.3.2 Left_arm.mod - Left Arm Program

Location: Main project folder

Key Parameters:

- Port: 1026
- Speed: v1500 mm/s

6 Tuning Image Processing Algorithms

6.1 bineralization Parameters

6.1.1 Threshold - Binarization Parameters

Location: `image_processor.py`, function `detect_edges_threshold()`

Key Parameters:

- `threshold_value` - Cutoff value (default 127)
- `max_value` - Maximum value for pixels above the threshold

How to Change:

1. Open the `image_processor.py` file
2. Find the `detect_edges_threshold()` function
3. Modify the `threshold_value`

Effect on the System:

- Lower value - more edges detected
- Higher value - fewer edges detected
- The optimal value depends on the photo's lighting

6.1.2 Canny - Detection Parameters

Location: `image_processor.py`, function `detect_edges_canny()`

Key Parameters:

- `min_threshold` - Minimum threshold (default 50)
- `max_threshold` - Maximum threshold (default 150)

How to Change:

1. Open the `image_processor.py` file
2. Find the `detect_edges_canny()` function
3. Adjust the threshold values

Effect on the System:

- Smaller difference between thresholds - more detail
- Larger difference between thresholds - less detail
- Chosen experimentally for a given image type

6.1.3 Adaptive - Adaptive Parameters

Location: `image_processor.py`, function `detect_edges_adaptive()`

Key Parameters:

- `block_size` - Neighborhood block size (default 11)
- `C` - Constant subtracted from the mean (default 2)

Effect on the System:

- Larger block_size - smoother transitions
- Smaller block_size - more detailed detection
- Parameter C affects detection sensitivity

6.2 Contour Simplification Parameters

6.2.1 epsilon Parameter for ‘approxPolyDP‘

Location: `image_processor.py`, function `simplify_contours()`

Default Value: 0.01 (1% of contour length)

How to Change:

1. Open the `image_processor.py` file
2. Find the ‘cv2.approxPolyDP()‘ call
3. Modify the epsilon value

Effect on the System:

- Smaller epsilon - more points, more accurate contours
- Larger epsilon - fewer points, simpler contours
- Too small epsilon slows down drawing
- Too large epsilon loses detail

6.2.2 Minimum Contour Length

Location: `image_processor.py`, function `filter_contours()`

Default Value: 10 pixels

How to Change:

1. Open the `image_processor.py` file
2. Find the contour filtering condition
3. Adjust the minimum length value

Effect on the System:

- Smaller value - more small details
- Larger value - cleaner image, less noise
- Depends on the resolution of the input image

6.3 Path Smoothing Parameters

6.3.1 Bézier Curve Parameters

Location: `coordinate_transformer.py`, function `smooth_path()`

How to Change:

1. Open the `coordinate_transformer.py` file
2. Find the `smooth_path()` function
3. Modify the interpolation parameters

Effect on the System:

- Smooths sharp corners in contours
- Improves the fluidity of robot movement
- May change the shape of details

7 Modifying Robot Coordinates and Safety

7.1 Dual-Arm Safety Parameters

7.1.1 Collision Buffer

Location: `simple_gui.py`, variable `forbidden_buffer`

Default Value: 40 mm

How to Change:

1. In the interface, go to Settings
2. Find the "Dual-arm Settings" section
3. Adjust the buffer value

Effect on the System:

- Smaller buffer - arms can work closer together
- Larger buffer - greater safety distance
- Too small a buffer increases the risk of collision
- Too large a buffer reduces the effective work area

7.1.2 Forbidden Zone for the Left Arm

Location: `coordinate_transformer.py`, function `master_slave_assign_contours()`

Default Coordinates: (0,0) to (130,40) mm

How to Change:

1. Open the `coordinate_transformer.py` file
2. Find the forbidden zone definition
3. Adjust the coordinates to the robot's geometry

Effect on the System:

- Must correspond to the robot's actual geometry
- Too small a zone increases the risk of collision with the base
- Too large a zone reduces the usable area

7.2 Robot Movement Parameters

7.2.1 Movement Speeds

Location: RAPID files (`Right_arm.mod`, `Left_arm.mod`)

Default Values:

- Drawing: v1500 mm/s
- Traveling: v1500 mm/s

How to Change:

1. Open the RAPID file for the appropriate arm
2. Find the speed definitions
3. Adjust the values (use caution!)

7.2.2 Positioning Precision

Location: RAPID files

Default Values:

- Move zone: z0
- Tolerance: 0.1 mm

How to Change:

1. Open the RAPID file
2. Find the ‘`move_zone`‘ definitions
3. Adjust to quality requirements

7.3 Coordinate System Parameters

7.3.1 Coordinate System Origin

Location: `simple_gui.py`, variable `use_center_origin`

Values: True (center), False (corner)

How to Change:

1. In the interface, go to Settings
2. Find the "Use Center Origin" option
3. Select the preferred system

Effect on the System:

- Center origin - origin is in the center of the drawing area
- Corner origin - origin is in the bottom-left corner
- Affects the interpretation of coordinates in RAPID programs

7.3.2 Coordinate Scaling

Location: `coordinate_transformer.py`, function `transform_paths()`

How to Change:

1. Open the `coordinate_transformer.py` file
2. Find the coordinate scaling function
3. Adjust the scaling factors

Effect on the System:

- Must correspond to the actual dimensions of the working area
- Incorrect scaling causes positioning errors
- Important when changing paper size

8 Debugging and Troubleshooting

8.1 Debugging Robot Connection

8.1.1 Checking Robot Availability

Tools:

- Command: ‘ping 192.168.125.1‘
- Checks the availability of the robot controller
- Diagnoses network problems

8.1.2 Testing Communication Ports

Location: `robot_communication.py`

How to Debug:

1. Add logging in the ‘connect()‘ function
2. Check if the socket opens
3. Test sending simple commands

8.1.3 Debugging Timeouts

Location: `robot_communication.py`, constant `START_COMMAND_TIMEOUT`

Default Value: 90 seconds

How to Change:

1. Open the `robot_communication.py` file
2. Find the timeout definition
3. Increase the value for slower connections (but this is already a lot of time, it’s for positioning if the robot had to find the paper itself, as it did before)

8.2 Debugging Image Processing

8.2.1 Visualizing Processing Steps

Location: `robot_drawer.py`, method `show_processing_steps()`

How to Enable:

1. Open the `robot_drawer.py` file
2. Uncomment the ‘`show_processing_steps()`‘ call
3. Run image processing

8.2.2 Saving Intermediate Images

Location: `image_processor.py`

How to Add:

1. At key points, add ‘cv2.imwrite()‘
2. Save images after each processing stage
3. Compare with expected results

8.3 Debugging Robot Movement

8.3.1 Logging Commands Sent to the Robot

Location: `robot_communication.py`

How to Enable:

1. Log all sent commands
2. Check the movement sequences

8.3.2 Testing Single Movements

Location: `robot_communication.py`, method `send_move()`

How to Test:

1. Create a simple test script
2. Test movements in safe coordinates
3. Check the robot’s responses

8.4 Debugging Dual-Arm Mode

8.4.1 Visualizing Contour Assignment

Location: `coordinate_transformer.py`

How to Debug:

1. The visualization is in settings->input methods->open detailed path viewer->animate dual-arm paths

8.4.2 Testing Arm Synchronization

Location: `robot_communication.py`, method `draw_paths_dual()`

How to Debug:

1. Add logging for the command sequences
2. Check execution times for both arms
3. Test WAIT synchronization commands

9 Extending System Functionality

9.1 Adding New bineralization Methods

9.1.1 Implementing a New Method

Location: `image_processor.py`

Implementation Template:

1. Add the function `‘detect_edges_[name]()‘`
2. Implement the detection algorithm
3. Add handling in the main function `‘load_and_process_image()‘`
4. Update the method selection interface

9.1.2 Integrating with the Interface

Location: `simple_gui.py`

How to Add:

1. Add a new value to `‘detection_method‘`
2. Update the selection menu in Settings
3. Add handling in the processing function

9.2 Adding New Voice Commands

9.2.1 Expanding the Command Dictionary

Location: `voice_commands.py`

How to Add:

1. Add a new command to the recognition dictionary
2. Implement handling in the function `‘_handle_command()‘` in `simple_gui.py`
3. Update the command documentation

9.2.2 Implementing the Command Action

Location: `simple_gui.py`, method `_handle_command()`

How to Add:

1. Add a case for the new command
2. Implement the operational logic
3. Add error handling

9.3 Adding New Drawing Styles

9.3.1 Implementing the Style in OpenAI

Location: `convert_to_lineart.py`

How to Add:

1. Add a new prompt for the style
2. Adjust the generation parameters
3. Test the quality in

10 System Pipeline Overview

This section provides an extremely detailed, step-by-step breakdown of the entire function call pipeline in the Robot Drawing System project, from user interaction through robot execution completion.

10.1 System Architecture Overview

The system consists of 6 main modules:

- `simple_gui.py`: Main GUI interface and orchestration
- `robot_drawer.py`: High-level drawing orchestration
- `image_processor.py`: Image processing and bineralization
- `coordinate_transformer.py`: Coordinate transformation and optimization
- `robot_communication.py`: Low-level robot TCP/IP communication
- `voice_commands.py`: Voice recognition and command processing
- `visualizer.py`: Data visualization and plotting

10.2 Detailed Pipeline Flow

10.2.1 Phase 1: Application Initialization

GUI Startup (`simple_gui.py`)

1. `SimpleRobotGUI.__init__()`:
    - Creates Tkinter root window
    - Loads configuration from `robot_gui_config.json` via `_load_config()`
    - Initializes `RobotDrawer` instance with default parameters (290x210mm workspace, center origin, TSP enabled)
    - Sets up voice command listener with `VoiceCommandListener`
    - Calls `create_simple_interface()` to build UI

- Starts connection status update loop with

  `root.after(100, self._update_connection_display)`

Voice System Initialization (`voice_commands.py`)

1. `VoiceCommandListener.__init__()`:
   - Sets up VOSK model path (handles both development and packaged environments)
   - Initializes audio queue and threading components
   - Prepares callback system for recognized commands

10.2.2 Phase 2: Image Acquisition

Camera/Image Input Methods **Method A: Robot Camera** (`simple_gui.py`)

1. User clicks "1. TAKE PICTURE" → `get_picture_from_robot()`
2. `get_picture_from_robot()` → `ready_robot_without_camera()` →

   `RobotController.start_without_camera()`
3. `RobotController.start_without_camera()` → `_send_command("READY\n")` to robot
4. Robot responds with "OK" after initialization
5. `get_picture_from_robot()` → `_get_picture_thread()` (background thread)
6. `_get_picture_thread()` → `RobotController.send_start()` → `_send_command("START\n")` or `START_CORNER\n` based on coordinate system
7. Robot performs positioning and sends "OK"
8. `RobotController.send_start()` returns success
9. `_get_picture_thread()` → gets picture from camera `robot_ftp_downloader.py`
10. `_load_robot_image()` processes downloaded image
11. `auto_process_image()` → `_start_auto_processing()` → `process_image()`

Method B: File Upload (`simple_gui.py`)

1. User clicks file selection → `browse_image()`
2. `filedialog.askopenfilename()` opens file picker
3. `browse_image()` → `load_preview_image()` → `PIL.Image.open()` loads image
4. `load_preview_image()` → `ImageTk.PhotoImage()` creates Tkinter-compatible image
5. `preview_label.configure(image=photo_image)` displays preview
6. `auto_process_image()` → `_start_auto_processing()` → `process_image()`

Method C: Text-to-Image Generation (`simple_gui.py`)

1. User enters text → `quick_text_generate()` → `_text_generation_thread()` (background)
2. `_text_generation_thread()` → AI image generation (implementation not shown)
3. Success → `_text_generation_success()` → `_auto_process_generated_image()`

Method D: Canvas Drawing (`simple_gui.py`)

1. User draws on canvas → mouse events trigger `start_canvas_drawing()`, `draw_on_canvas()`, `stop_canvas_drawing()`
2. `save_drawing()` → `PIL.ImageGrab.grab()` captures canvas
3. `use_drawing()` → `load_preview_image()` processes captured image

10.2.3 Phase 3: Image Processing Pipeline

Main Processing (`simple_gui.py` → `robot_drawer.py` → `image_processor.py`)

1. `process_image()` → `_process_thread()` (background processing)
2. `_process_thread()` → `RobotDrawer.load_image()` with current settings:
   - `precision`: "high", "medium", "low", etc.
   - `enable_tsp`: Boolean from GUI checkbox
   - `detection_method`: "canny", "threshold", "adaptive"
   - `logo_settings`: Dictionary with enabled/corner/size
   - `protect_logo`: True for logo processing

Image Processing Details (`image_processor.py`)

1. `ImageProcessor.load_and_process_image()` → `extract_edge_following_path()`
2. `extract_edge_following_path()`:
   - `cv2.imread()` loads image
   - `cv2.rotate(ROTATE_180)` flips image 180°
   - `cv2.mirror` mirrors it horizontally
   - **Border Padding**: `cv2.copyMakeBorder()` adds white padding (unless `protect_logo=True`)
   - `cv2.cvtColor(BGR2GRAY)` converts to grayscale
   - `cv2.GaussianBlur()` + `cv2.medianBlur()` noise reduction
3. **Bineralization Methods**:
   - **Canny**: `cv2.Canny()` → `_thin_edges()` morphological thinning → `cv2.morphologyEx(CLOSE)`
   - **Threshold**: `cv2.threshold(THRESH_BINARY + THRESH_OTSU)`

- **Adaptive**: `cv2.adaptiveThreshold(ADAPTIVE_THRESH_GAUSSIAN_C)` with precision-based parameters

4. **Contour Extraction**:
   - `cv2.findContours(RETR_LIST, CHAIN_APPROX_SIMPLE)` finds contours
   - **Filtering**: Remove contours < `MIN_CONTOUR_LENGTH` (10) or < area threshold (adaptive)
   - **Border Filtering**: `_is_border_contour()` removes contours touching image edges (unless `protect_logo=True`)
5. **Contour Simplification**:
   - `cv2.approxPolyDP()` with adaptive epsilon based on precision and image properties
   - Resolution-based factor: 1000000px → 3.0, 500000px → 2.0, etc.
   - Scale factor normalization: `min(2.0, max(0.5, scale * 1000))`
   - Minimum epsilon: 0.5px, Maximum: perimeter * 0.1
6. **Path Optimization** (if `enable_tsp=True`):
   - Contours sorted by length (longest first)
   - Returns `{'contours': simplified_paths, 'image_shape': edges.shape, 'simplification_factor': factor}`

Coordinate Transformation (`coordinate_transformer.py`)

1. `CoordinateTransformer.contours_to_robot_coordinates()`:
   - Calculates scale: `min(scale_x, scale_y)` where `scale_x = effective_x / width`
   - `effective_x = max_x - 2*margin_x`, `effective_y = max_y - 2*margin_y`
   - Centers image: `final_offset_x = effective_offset_x + margin_x`
   - **Coordinate System Conversion**:
     - **Center Origin**: `robot_x = (x * scale) + final_offset_x - (max_x / 2)`
     - **Corner Origin**: `robot_x = (x * scale) + final_offset_x`
     - Y-axis flip: `robot_y = ((height - y) * scale) + final_offset_y - (max_y / 2)`
2. **Path Smoothing** (if enabled):
   - **Bezier**: `_smooth_path_bezier()` with quadratic curves
   - **Catmull-Rom**: `_smooth_path_catmull_rom()` with cubic splines

3. **Point Filtering**: `_filter_min_distance()` removes points closer than `MIN_DISTANCE_THRESHOLD` (1.0mm)

Logo Processing (if enabled)

1. `RobotDrawer.generate_logo_coordinates()`:
   - Loads `logo_short.png`
   - `ImageProcessor.extract_edge_following_path()` with `protect_logo=True`
   - Scales logo to `size_mm` while maintaining aspect ratio
   - Positions at top-right corner with 5mm margin
   - **Logo Flipping**: `x = logo_x_offset + logo_width_mm - (px * scale_factor)` (horizontal mirror)
   - Coordinate system conversion (center/corner origin)
   - Minimum distance filtering
   - Appends logo contours to main drawing points

10.2.4 Phase 4: Style Processing (Portrait/Carricature)

Portrait Mode (`simple_gui.py` → `convert_to_lineart.py`)

1. User selects portrait → `set_style_and_convert("portrait")`
2. `convert_to_face_drawing()` → `_face_drawing_thread()` (background)
3. `_face_drawing_thread()` → AI face drawing generation
4. Success → `_face_drawing_success()` → `load_preview_image()` → `auto_process_image()`

Caricature Mode (`simple_gui.py` → `convert_to_lineart.py`)

1. User selects caricature → `set_style_and_convert("caricature")`
2. `convert_to_caricature()` → `_caricature_thread()` (background)
3. `_caricature_thread()` → AI caricature generation
4. Success → `_caricature_success()` → `load_preview_image()` → `auto_process_image()`

10.2.5 Phase 5: Robot Connection and Preparation

Connection Management (`simple_gui.py` → `robot_communication.py`)

1. User clicks connection toggle → `toggle_connection()`
2. `toggle_connection()` → `_connect_thread()` (background)
3. `_connect_thread()` → `RobotController.connect()`
4. `RobotController.connect()`:
   - `socket.socket(AF_INET, SOCK_STREAM)` creates sockets

- `socket.connect((ip, port))` connects to port 1025 (right arm)
- `socket.connect((ip, port_l))` connects to port 1026 (left arm)
- Returns success status

5. **Robot Initialization**
6. `RobotController.send_start()` or `start_without_camera()`:
   - Sends "START\n" or "START_CORNER\n" based on coordinate system
   - Robot performs: positioning
   - Robot responds "OK" after ~2 seconds

10.2.6 Phase 6: Drawing Execution

Single-Arm Drawing (`simple_gui.py` → `robot_drawer.py` → `robot_communication.py`)

1. User clicks "4. START DRAWING" → `start_drawing()` → `start_robot_drawing()`
2. `start_robot_drawing()` → `RobotDrawer.draw()` with progress callback
3. `RobotDrawer.draw()` → `RobotController.draw_paths()` with batch mode
4. `RobotController.draw_paths()` → `_draw_with_ultra_fast_mode()` (if batch enabled)

Batch Drawing (`robot_communication.py`)

1. `_draw_with_ultra_fast_mode()`:
   - **Pen Management**: `send_pen_up()` (already up), `time.sleep(0.02)`
   - **Per Contour Loop**:
     - `should_stop()` check for emergency stop
     - `_transition_to_start()`: Horizontal U-shape movement
       - `send_between_command("RETREAT\n")` moves back horizontally (±80mm X)
       - `send_move()` to Y-level of next contour start
       - `send_move()` forward to the actual start position
     - `send_pen_down()`, `time.sleep(0.02)`
     - `send_move()` to first point with pen down
     - **Batch Processing**: `send_batch_moves_ultra_fast()` for remaining points
       - Groups points into batches (dynamic sizing based on coordinate length)
       - `send_batch_moves()` → `_send_command(f"BATCH,{coords}\n")`
       - 80-character command limit handling with batch splitting
     - `send_pen_up()`, `send_between_command(next_start)` for retreat

- **Progress Tracking**: Updates via `progress_callback(current_batch, total_batches, points_sent, total_points)`

Dual-Arm Drawing (`robot_drawer.py` → `robot_communication.py`)

1. `RobotDrawer.draw_dual()` (alternative path):
   - `master_slave_assign_contours()` assigns contours to right/left arms
   - Creates forbidden zones around master contour with circular buffers + tail extension
   - `RobotController.draw_paths_dual()` with threading
   - **Threaded Execution**: `t_right` and `t_left` threads with `threading.Barrier(2)` synchronization
   - **Per-Step Processing**:
     - Collision avoidance: `_conflicts_with_peer()`, `_wait_for_peer_retreat()`
     - `_transition_to_start()` with side-specific X offsets
     - Simultaneous contour drawing with barrier synchronization
     - WAIT commands for synchronization between arms

10.2.7 Phase 7: Voice Command Processing

Voice Recognition (`voice_commands.py`)

1. `VoiceCommandListener.start()` → `_run()` in background thread
2. `sd.RawInputStream()` captures audio at 16kHz
3. `KaldiRecognizer.AcceptWaveform()` processes audio chunks
4. `json.loads(result)` extracts recognized text
5. If text in `commands` list → `callback(text)`

Command Dispatch (`simple_gui.py`)

1. `_on_voice_command()` → `_handle_command()` (dispatched to main thread)
2. **Command Mapping**:
   - "połącz" → `toggle_connection()`
   - "start" → `start_drawing()`
   - "stop" → `emergency_stop()`
   - "akcja" → `get_picture_from_robot()`
   - "kamera" → `show_camera_preview()`
   - "portret" → portrait selection
   - "karykatura" → caricature selection

- "podgląd" → `open_detailed_path_window()`

10.2.8 Phase 8: Visualization and Monitoring

Preview Systems (`visualizer.py`)

1. `DrawingVisualizer.plot_preview()`:
   - `matplotlib.pyplot.figure()` creates plot
   - Plots each contour with different colors
   - Sets up centered coordinate system: `xlim(-max_x/2, max_x/2)`
   - `invert_yaxis()` so (0,0) is top-left
   - Draws workspace border and center markers
2. `show_processing_steps()`:
   - Three-panel plot: Original → Edges → Robot Preview
   - `cv2.cvtColor(BGR2RGB)` for proper color display

Real-time Progress (`simple_gui.py`)

1. Progress callbacks update:
   - `bottom_progress_bar['value']` for visual progress
   - `progress_indicator` text updates
   - Status messages in `status_text`

10.2.9 Phase 9: Error Handling and Cleanup

Error Scenarios

1. **Connection Failures**: `RobotController.connect()` → `_connection_failed()` → UI updates
2. **Processing Errors**: `_process_failed()` → error dialogs
3. **Drawing Errors**: `_draw_failed()` → cleanup commands
4. **Emergency Stop**: `emergency_stop()` → `RobotController.send_stop()` on both arms

Cleanup Operations

1. `SimpleRobotGUI._on_close()`:
   - `VoiceCommandListener.stop()`
   - `RobotController.disconnect()` → `send_stop()` → `socket.close()`
   - Configuration saving via `_save_config()`

11 Settings Explanations

This section provides detailed explanations for every setting available in the application's Settings window (accessible via the Settings button). The settings are organized by tab for easy reference.

11.1 Input Methods Tab

11.1.1 Camera Index

Description: Selects which camera device to use for photo capture.
Options: Integer values (0, 1, 2, etc.) representing different camera devices connected to your computer.
Default: 0 (usually the built-in webcam).
Usage: If you have multiple cameras (e.g., external USB webcam), change this to select the desired one. Use the "Test Camera" button to verify the selection.
Impact: Affects which camera is used for taking photos in the main interface.

11.1.2 Test Camera Button

Description: Opens a preview window to test the selected camera.
Usage: Click to verify the camera is working and adjust positioning before taking photos.
Impact: Helps ensure the camera is properly configured before use.

11.2 Processing Tab

11.2.1 Quality

Description: Controls the precision of image processing and contour simplification.
Options: High, Medium, Low.
Default: High.
Usage: Higher quality preserves more detail but takes longer to process and may result in more complex robot paths. Lower quality is faster but may lose fine details.
Impact: Affects processing time, contour accuracy, and robot drawing complexity.

11.2.2 Detection Method

Description: Chooses the algorithm for binarization edges in the image.
Options: Threshold, Canny, Adaptive.
Default: Adaptive.
Usage: Threshold is simple and fast for good lighting. Canny detects edges based on brightness changes. Adaptive automatically adjusts parameters for varying lighting conditions.
Impact: Determines how well the system identifies drawable contours from the image.

11.2.3 Use Batch Mode

Description: Enables sending multiple movement commands to the robot at once.
Options: Checked/Unchecked.
Default: True.

Usage: Batch mode groups commands for faster transmission. Disable only if experiencing communication issues.
Impact: Significantly speeds up drawing by reducing network overhead.

11.2.4 Use Center Origin

Description: Sets the coordinate system origin.
Options: Checked/Unchecked.
Default: True.
Usage: When checked, (0,0) is at the center of the drawing area. When unchecked, (0,0) is at the bottom-left corner.
Impact: Affects how coordinates are interpreted by the robot's RAPID program.

11.2.5 Enable Logo

Description: Adds a logo to the drawing.
Options: Checked/Unchecked.
Default: False.
Usage: When enabled, a logo is automatically added to the top-right of the drawing.
Impact: Personalizes the output but may increase processing time.

11.2.6 Logo Size

Description: Sets the size of the logo in millimeters.
Options: Numeric value (e.g., 20).
Default: 20.
Usage: Adjust based on desired logo prominence in the final drawing.
Impact: Larger logos may cover more of the drawing area.

11.2.7 Enable Frame Filtering

Description: Removes contours that touch the image edges.
Options: Checked/Unchecked.
Default: False.
Usage: Enable to clean up images with borders or frames that shouldn't be drawn.
Impact: Reduces unwanted contours from image borders.

11.3 Connection Tab

11.3.1 Robot IP

Description: The IP address of the ABB YuMi robot controller.
Options: Valid IP address (e.g., 192.168.125.1).
Default: 192.168.125.1.
Usage: Enter the IP address provided by your robot's network configuration.
Impact: Required for establishing communication with the robot.

11.3.2 Robot Port

Description: The TCP port for the right arm of the robot.
Options: Numeric port number.
Default: 1025.
Usage: Usually 1025 for the right arm. Change only if your robot uses different ports.
Impact: Must match the robot's RAPID program configuration.

11.3.3 Robot Port L

Description: The TCP port for the left arm of the robot.
Options: Numeric port number.
Default: 1026.
Usage: Usually 1026 for the left arm. Change only if your robot uses different ports.
Impact: Must match the robot's RAPID program configuration for dual-arm mode.

11.4 Drawing Tab

11.4.1 Max X

Description: Maximum width of the drawing area in millimeters.
Options: Numeric value.
Default: 290.
Usage: Set to match your paper or drawing surface width.
Impact: Defines the horizontal bounds of the robot's workspace.

11.4.2 Max Y

Description: Maximum height of the drawing area in millimeters.
Options: Numeric value.
Default: 210.
Usage: Set to match your paper or drawing surface height.
Impact: Defines the vertical bounds of the robot's workspace.

11.4.3 Margin X

Description: Horizontal safety margin in millimeters.
Options: Numeric value.
Default: 10.
Usage: Increases to add more space from edges, decreases for larger drawing area.
Impact: Prevents the robot from drawing too close to the edges.

11.4.4 Margin Y

Description: Vertical safety margin in millimeters.
Options: Numeric value.
Default: 10.
Usage: Increases to add more space from edges, decreases for larger drawing area.
Impact: Prevents the robot from drawing too close to the edges.

11.4.5 Brush Size

Description: Size of the drawing tool in millimeters.
Options: Numeric value.
Default: 3.
Usage: Adjust based on your pen or marker size for accurate path planning.
Impact: Affects spacing between drawing points.

11.4.6 Dual Arm Mode

Description: Enables drawing with both robot arms simultaneously.
Options: Checked/Unchecked.
Default: True.
Usage: Enable for faster drawing with two arms. Disable for single-arm operation.
Impact: Requires proper robot configuration and increases drawing speed.

11.4.7 Forbidden Buffer

Description: Safety buffer around each arm in dual-arm mode.
Options: Numeric value in millimeters.
Default: 40.
Usage: Increase for more safety, decrease for closer arm operation.
Impact: Prevents arm collisions in dual-arm mode.

11.5 Advanced Tab

11.5.1 Text Prompt

Description: Default text prompt for AI image generation.
Options: Text string.
Default: Empty.
Usage: Enter a base prompt that will be used when generating images from text.
Impact: Provides a starting point for text-to-image conversion.

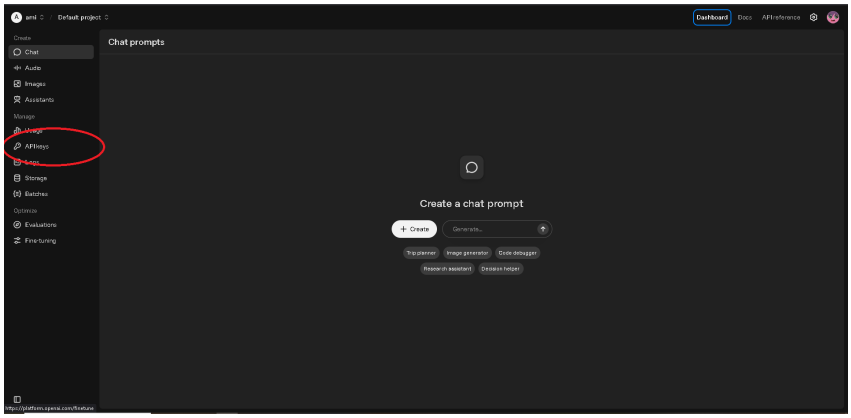
12 API Configuration and Setup

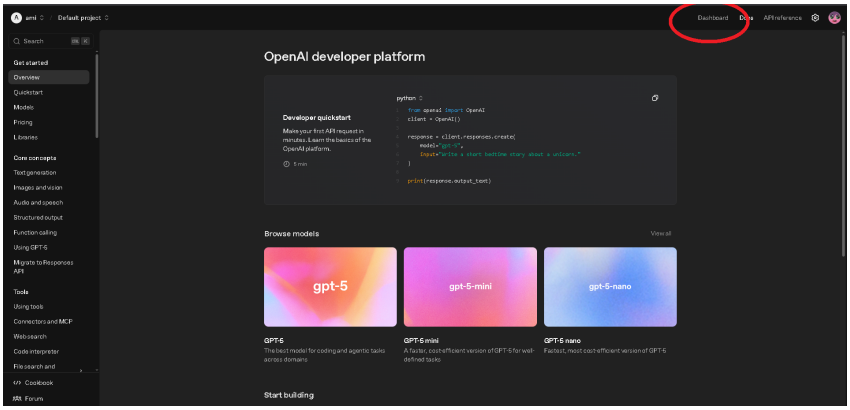
12.1 Changing the OpenAI API Key

The application uses OpenAI's API for image generation (portrait and caricature modes). To use this feature, you need a valid OpenAI API key.

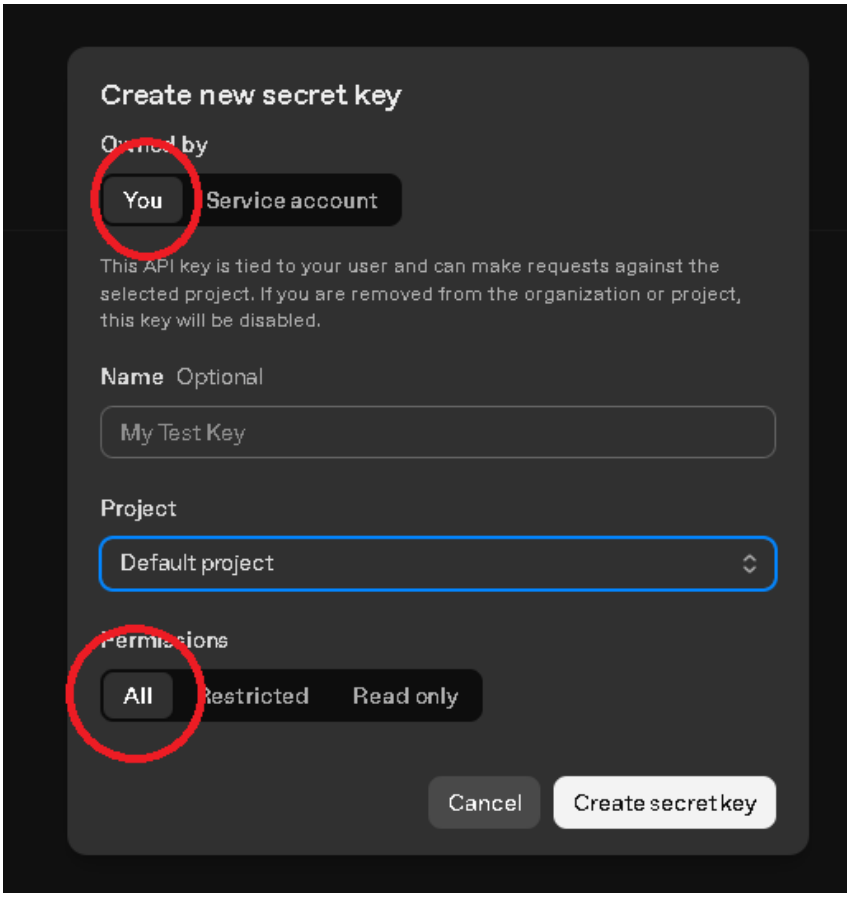
12.1.1 How to Get an OpenAI API Key

1. Go to `https://platform.openai.com/`
2. Sign up or log in to your OpenAI account
3. Go to dashboard

H 



4. Go to API keys section
5. Click "Create new secret key" and use those settings:



6. Copy the generated key

12.1.2 How to Set the API Key in the Application

1. Open the `convert_to_lineart.py` file in your project
2. Look for the line where the API key is defined
3. Replace the key with your own API key
4. Save the file

12.1.3 Troubleshooting API Issues

- **Error: Invalid API Key** - Double-check that you've copied the key correctly
- **Error: Insufficient Credits** - Check your OpenAI account balance
- **Error: Rate Limit Exceeded** - Wait a few minutes before trying again

12.2 Compiling to Executable (.exe)

To create a standalone executable from your Python code:

12.2.1 Using PyInstaller

1. Ensure PyInstaller is installed: `pip install pyinstaller`

2. Run the build command:

   `pyinstaller.exe .\RobotDrawingSystem.spec`

3. The executable will be created in the `dist` folder as `RobotDrawingSystem.exe`

12.2.2 What the .spec File Does

The `RobotDrawingSystem.spec` file contains:

- Main script: `simple_gui.py`
- Included data files: `logo_inlader.jpg`, `vosk-model-small-pl-0.22`
- Hidden imports: `vosk`, `vosk.Model`, etc.
- Build options: onefile, console=False, icon settings

12.2.3 Customizing the Build

- **Add files:** Edit the `datas` list in the .spec file
- **Change icon:** Replace `insta.png` with your icon file
- **Debug mode:** Set `debug=True` for troubleshooting